



Read Chapters 2, 4 and 5

Unit 1_01

Java Fundamentals

The Java Environment

Introduction to Java

Outline



- **What is Java?**
- **Java versions**
- **Java development**
- **Java portability**
- **Types of errors**

What is Java?

- Java is a class-based, general-purpose, **object-oriented** programming language.
- Originally developed by James Gosling at Sun Microsystems (1995)
  - Oracle acquired SUN Microsystems (2010)
- Syntax is similar to C and C++ with **fewer low-level facilities**
- The latest version is Java **21** (2024)
- Very popular:
  - Billions of devices, 10 million developers.

Applications of Java

Mobile Applications

Desktop GUI Applications

Web-based Applications

Web Servers and Application Servers

Enterprise Applications

Scientific Applications

Gaming Applications

Big Data Technologies

Business Applications

Cloud-based Applications

Java Editions

- The software you use to write Java programs is called the **Java Development Kit**, or **JDK**. Editions of the JDK:
  - Java SE - Java2 Standard Edition.
  - Java EE - Java2 Enterprise Edition.
  - Java ME - Java2 Micro Edition.
- Available for download at

  http://java.oracle.com

Task 1

- ***By next class***
  - ***Download and install the JDK***
  - ***Install Eclipse and try basic programs of class 1***

Java Editions

- Java **Standard** Edition (Java SE) contains the capabilities needed to develop desktop and server applications.
- Java **Enterprise** Edition (Java EE) is geared toward developing:
  - large-scale applications
  - distributed networking applications
  - and web-based applications.
- Java **Micro** Edition (Java ME) Geared toward developing applications for resource-constrained embedded devices, such as
  - Smartwatches
  - MP3 players
  - television set-top boxes



Java Development

Normally there are five phases:

1. edit
2. compile
3. load
4. verify
5. execute.

A Typical Java Development Environment

- **Phase 1 Editing a file with an editor program**
  1. Using the editor, you type a Java program (source code).
  2. Make any necessary corrections.
  3. Save the program.
  4. Java source code files are given a name ending with the **.java** extension.
     - **Integrated development environments (IDEs)** provide tools that support the software development process.
     - The most popular Java IDEs are:
       - **Eclipse** (http://www.eclipse.org)
       - IntelliJ IDEA (http://www.jetbrains.com)
       - NetBeans (http://www.netbeans.org)

A Typical Java Development Environment

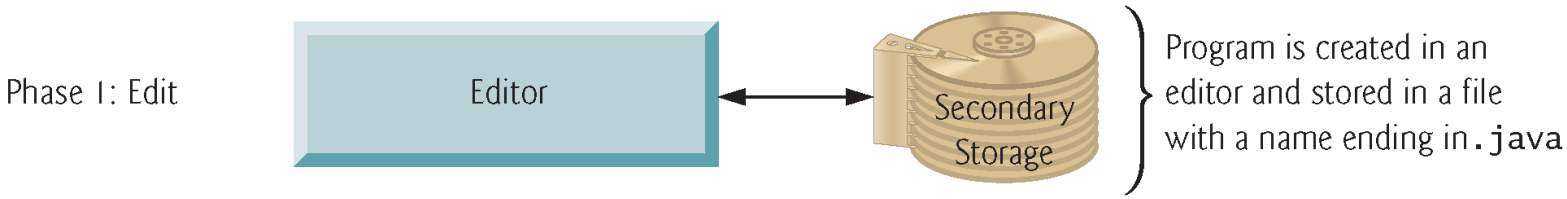


Fig. 1.6 | Typical Java development environment—editing phase.

A Typical Java Development Environment

- **Phase 2: Compiling a Java Program into Bytecodes**
- Use the command **javac** (the Java compiler) to compile a program. For example, to compile a program called ***Welcome.java***, you'd type

```
javac Welcome.java
```

- If the program compiles, the compiler produces a **.class** file called ***Welcome.class*** that contains the compiled version.

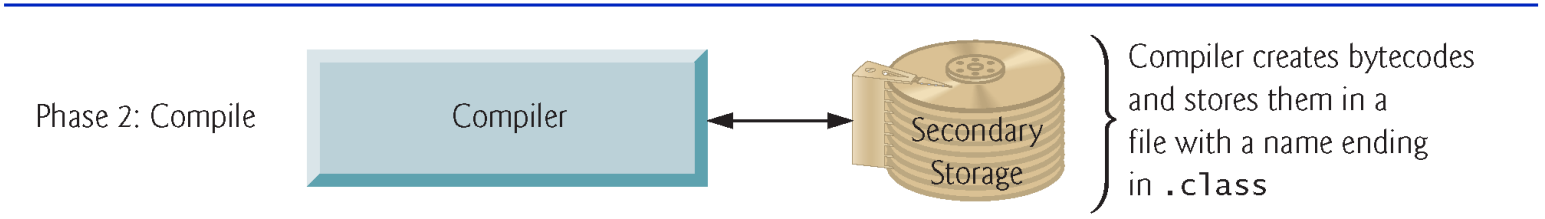


Fig. 1.7 | Typical Java development environment—compilation phase.

A Typical Java Development Environment

- **Bytecodes?**
- Java compiler translates Java source code into bytecodes
  - Bytecode instructions are platform independent
- The **Java Virtual Machine (JVM)—**a part of the JDK and the foundation of the Java platform—executes bytecodes.
- Virtual machine (VM)—**a software application that simulates a computer**
- Hides the underlying operating system and hardware from the programs that interact with it.
- The JVM is invoked by the java command. For example, to execute a Java application called ***Welcome***, you'd type the command

java Welcome

A Typical Java Development Environment

- **Phase 3: Loading a Program into Memory**
  - The **JVM** places the program in memory to execute it—this is known as loading.
  - **Class loader** takes the .class files containing the program's bytecodes and transfers them to primary memory. Also loads any of the .class files provided by Java that your program uses.
  - The .class files can be loaded from a disk on your system or over a network.

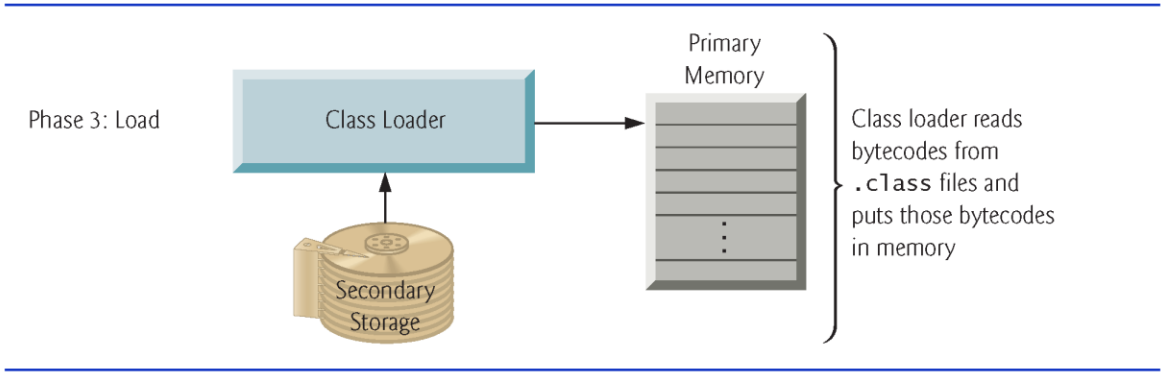


Fig. 1.8 | Typical Java development environment—loading phase.

A Typical Java Development Environment

- **Phase 4: Bytecode Verification**
  - The **bytecode verifier** examines their bytecodes ensuring that they're valid and do not violate Java's security restrictions.
  - Java enforces strong security to make sure that programs do not damage your files or your system (as computer viruses and worms might).

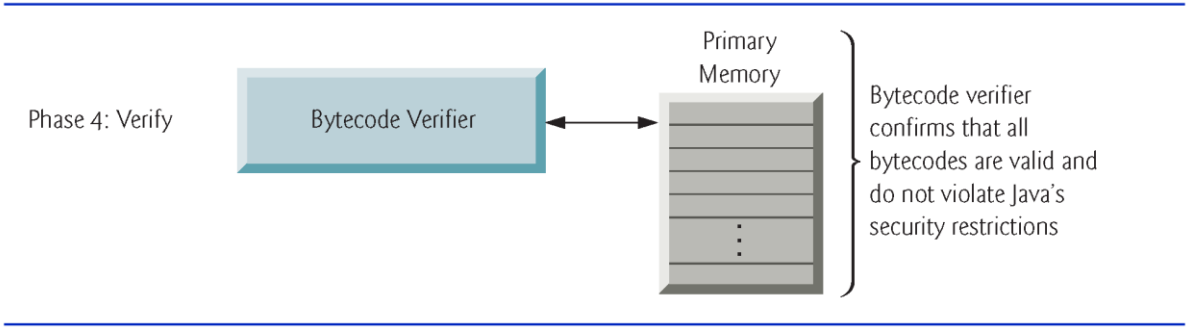


Fig. 1.9 | Typical Java development environment—verification phase.

A Typical Java Development Environment

- **Phase 5: Execution**
- JVMs typically ***execute*** bytecodes using a combination of **interpretation** and **so-called just-in-time (JIT) compilation**. Analyzes the bytecodes as they're interpreted.
- **A just-in-time (JIT) compiler translates the bytecodes into the underlying computer's machine language** for the actual computer on which the program.

ex

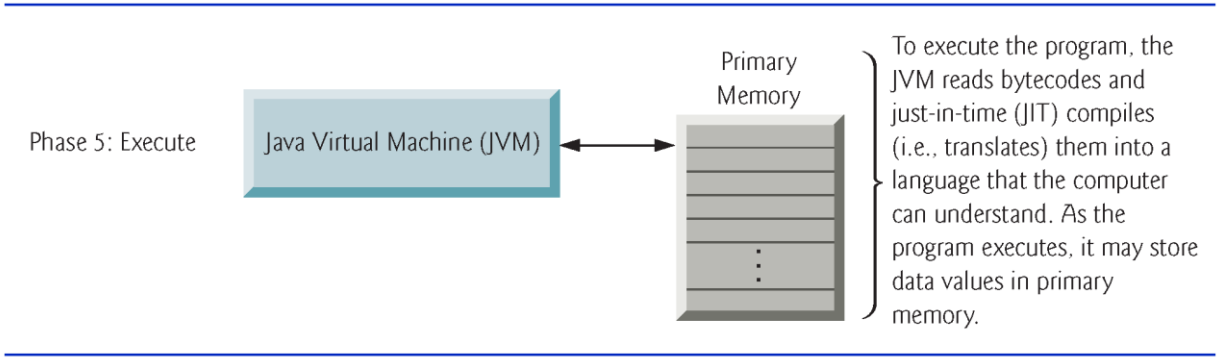
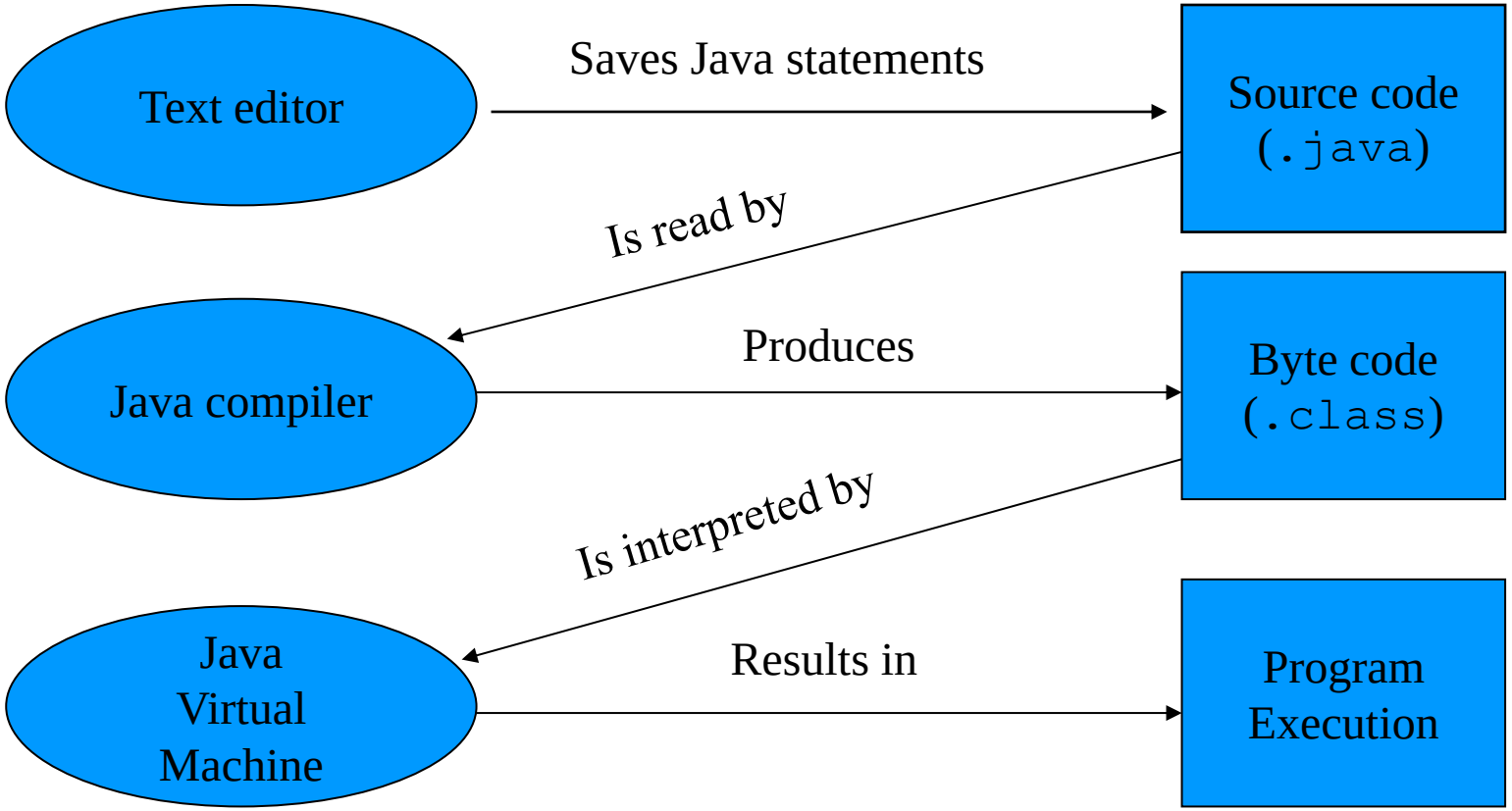
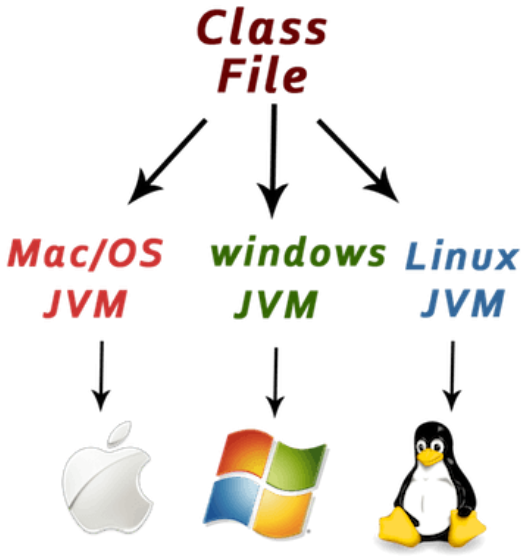


Fig. 1.10 | Typical Java development environment—execution phase.

Program Development Process

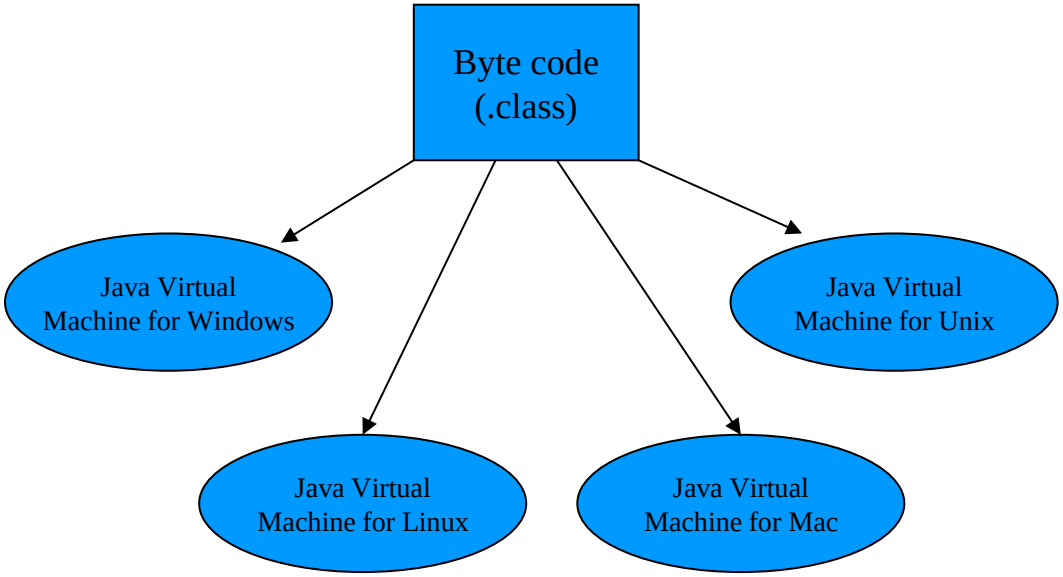


Java Portability



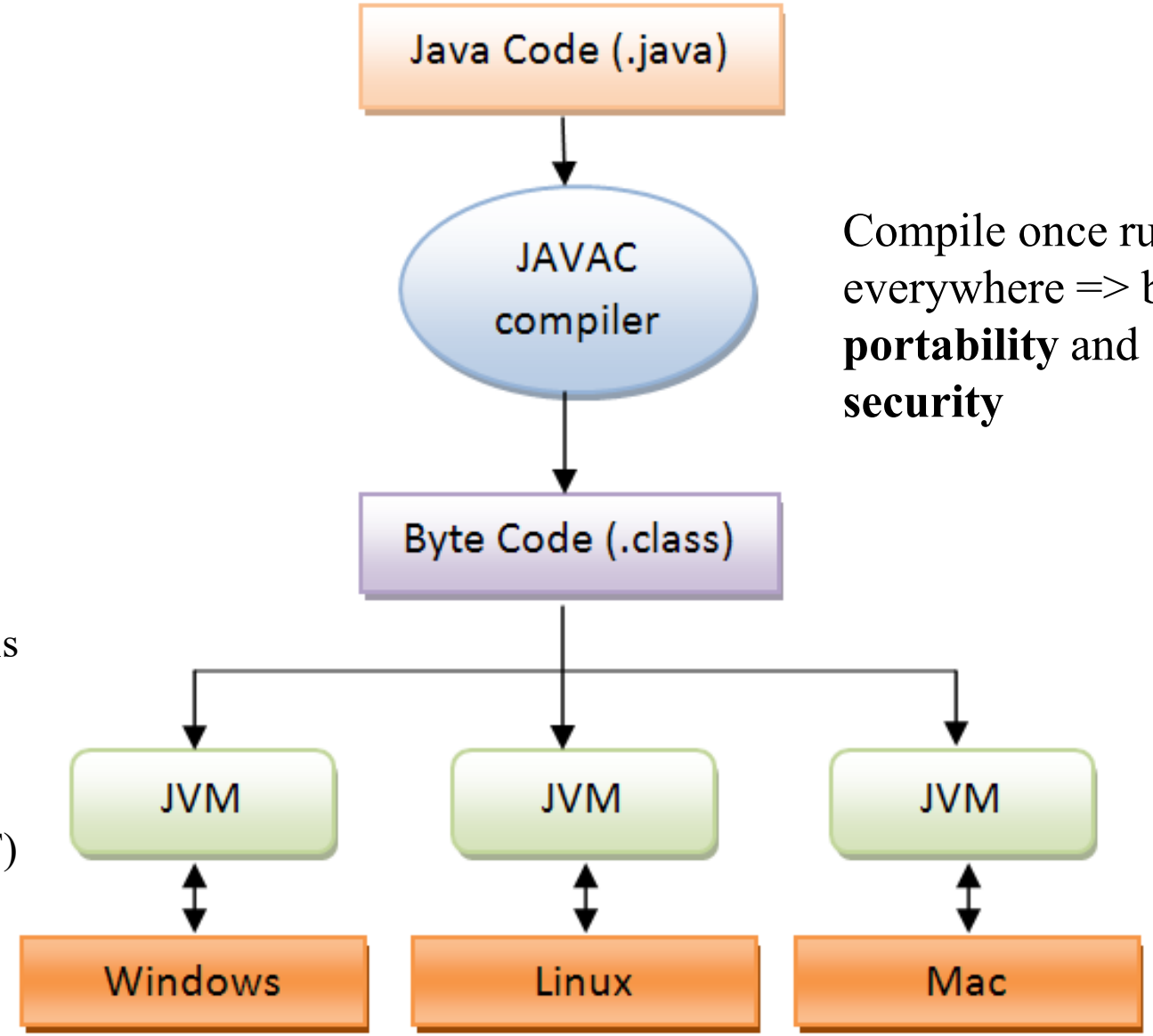
Portability in Java

- While most programming languages achieve portability by compiling a program for each CPU it will run on, Java provides an JVM for each platform so that programmers do not have to recompile for different platforms.



Java Compilation

- **Java** source code is **compiled** into bytecode (saved in .class file)



Compile once run everywhere => better **portability** and **security**

- When the program is to be run, the bytecode is converted, using the just-in-time (JIT) compiler, into executable machine code

Types of Errors



Syntax Errors

- ***Grammatical mistakes in a program***.
- The ***grammatical rules*** for writing a program are very strict.
- The compiler catches syntax errors and prints an error message.
- Detected during compilation.
- Example: using a period where a program expects a comma.

Runtime Errors

- Detected during run time.
- ***When the computer detects an error, it terminates the program and prints an error message.***
- Example: attempting to divide by 0

Logic Errors

- Errors that are not detected during compilation or while running, but which **cause the program to produce incorrect results.**

- **Example:**
- An attempt to calculate a Fahrenheit temperature from a Celsius temperature by multiplying by 9/5 and adding 23 instead of 32.